EXERCICE 10 · CODE COMPLET

Todo App

Tous les fichiers utilisés, en entier, dans l'ordre de lecture.

1. **correction** (4 fichiers)

1 correction

Fichiers générés par Vite, non modifiés (non affichés) : index.html, vite.config.js, .gitignore, eslint.config.js, public/vite.svg, package.json

FICHIER **correction/src/main.jsx** 6 lignes

```jsx
import { createRoot } from "react-dom/client";
import App from "./App.jsx";

createRoot(document.getElementById("root")).render(
  <App />
);
```

FICHIER **correction/src/App.jsx** 8 lignes

```jsx
import "./App.css";
import Todo from "./components/Todo";

function App() {
  return <Todo />;
}

export default App;
```

FICHIER **correction/src/App.css** 32 lignes

```css
/* VERSION RESPONSIVE - S'adapte à tous les écrans (mobile, tablette, desktop) */

/* Importation de la police Google Fonts Inter avec plusieurs graisses */
@import url("https://fonts.googleapis.com/css2?family=Inter:wght@400;500;600;700&display=swap");

/* Sélecteur universel qui cible tous les éléments HTML */
* {
  /* Force tous les éléments à inclure padding et border dans leur largeur/hauteur totale */
  box-sizing: border-box;
  /* Supprime les marges par défaut du navigateur pour avoir un contrôle total */
  margin: 0;
  /* Supprime les espacements internes par défaut du navigateur */
  padding: 0;
}

/* Style global du corps de la page */
body {
  /* Applique la police Inter importée, avec sans-serif comme police de secours */
  font-family: "Inter", sans-serif;
  /* Améliore le rendu des polices sur les navigateurs WebKit (Safari, Chrome) */
  -webkit-font-smoothing: antialiased;
  /* Améliore le rendu des polices sur Firefox pour Mac */
  -moz-osx-font-smoothing: grayscale;

  text-align: center;

  padding: 20px;
}
```

```
29
30 h1 {
31   padding: 20px;
32 }
```

FICHIER **correction/src/components/Todo.jsx** 135 lignes

```jsx
1  import { useEffect, useState } from "react";
2
3  export default function TodoApp() {
4    const [todos, setTodos] = useState([]);
5    const [filter, setFilter] = useState("all");
6    const [loading, setLoading] = useState(true);
7
8    // Calcul dérivé au lieu d'un état séparé
9    const getFilteredTodos = (todos, filter) => {
10     switch (filter) {
11       case "todo":
12         return todos.filter((t) => !t.completed);
13       case "done":
14         return todos.filter((t) => t.completed);
15       default:
16         return todos;
17     }
18   };
19
20   const filteredTodos = getFilteredTodos(todos, filter);
21
22   // Fonction de fetch
23   const fetchTodos = async () => {
24     try {
25       const res = await fetch(
26         "https://jsonplaceholder.typicode.com/todos?_limit=10"
27       );
28       const data = await res.json();
29       setTodos(data);
30     } catch (error) {
31       console.error("Error:", error);
32     } finally {
33       setLoading(false);
34     }
35   };
36
37   // Un seul useEffect pour charger les données
38   useEffect(() => {
39     fetchTodos();
40   }, []); // Une seule fois au mount
41
42   // useEffect avec dépendance
43   useEffect(() => {
44     console.log("Filter changed:", filter);
45   }, [filter]);
46
47   // Toggle pour modifier les todos
48   const toggleTodo = (id) => {
49     /*const nextTodos = [...todos];
50     const todo = nextTodos.find((t) => t.id === id);
51     if (todo) {
52       todo.completed = !todo.completed;
53       setTodos(nextTodos);
54     }*/
55
56     /*setTodos(
57       todos.map((t) => (t.id === id ? { ...t, completed: !t.completed } : t))
58     );*/
```

```
59
60      setTodos((prevTodos) =>
61        prevTodos.map((t) =>
62          t.id === id ? { ...t, completed: !t.completed } : t
63        )
64      );
65    };
66
67    return (
68      <div style={{ padding: 16, maxWidth: 600, margin: "0 auto" }}>
69        <h2>Todo App</h2>
70
71        {/* Filtres avec compteurs */}
72        <div
73          style={{
74            display: "flex",
75            gap: 8,
76            justifyContent: "center",
77            margin: 16,
78          }}
79        >
80          <button
81            onClick={() => setFilter("all")}
82            style={{ fontWeight: filter === "all" ? "bold" : "normal" }}
83          >
84            All ({todos.length})
85          </button>
86          <button
87            onClick={() => setFilter("todo")}
88            style={{ fontWeight: filter === "todo" ? "bold" : "normal" }}
89          >
90            Todo ({todos.filter((t) => !t.completed).length})
91          </button>
92          <button
93            onClick={() => setFilter("done")}
94            style={{ fontWeight: filter === "done" ? "bold" : "normal" }}
95          >
96            Done ({todos.filter((t) => t.completed).length})
97          </button>
98        </div>
99
100       {/* Liste ou Loading */}
101       {loading ? (
102         <div style={{ textAlign: "center" }}>Loading...</div>
103       ) : (
104         <ul style={{ padding: 0 }}>
105           {filteredTodos.map((todo) => (
106             <li key={todo.id} style={{ listStyle: "none", margin: 12 }}>
107               <label
108                 style={{
109                   cursor: "pointer",
110                   display: "flex",
111                   alignItems: "center",
112                 }}
113               >
114                 <input
115                   type="checkbox"
116                   checked={todo.completed}
117                   onChange={() => toggleTodo(todo.id)}
118                   style={{ marginRight: 8 }}
119                 />
120                 <span
121                   style={{
122                     textDecoration: todo.completed ? "line-through" : "none",
123                     opacity: todo.completed ? 0.6 : 1,
```

```
                  }}
                >
                  {todo.title}
                </span>
              </label>
            </li>
          ))}
        </ul>
      )}
    </div>
  );
}
```